

Subject: CS336, Spring, 2026

Date: from July 20, 2026 to August 12, 2026

Contents

Lecture 1: Overview and Tokenization

Reference: CS336, Lecture 1: Overview, Tokenization

Introduction

Overview

What's new? 本课程为 CS336 的第三次开设，与前面两次课程相比增加了更多现代 Language Model (LM) 的元素，例如 mixture of experts (MoE), long-context, agent 等。

Why this course? 本课程的宗旨依然是 *"Understanding via Building"*，虽然目前很多前沿研究以 prompt API models 为主，但理解 LM 的基础依然是对于基础研究非常必要。

What can we learn? ① **Mechanics:** 原理，例如 Transformer 是什么，如何让模型并行运行。② **Mindset:** 思维方式，例如最大化利用硬件，考虑 scaling law。③ **Intuitions:** 直觉，例如什么样的数据、模型可以获得更好的效果。

Bitter lesson. 目前模型的规模在不断扩大，一个错误的认识是：*"Scale is all that matters, algorithms don't matter."* 但正确的看法是：*"Algorithms that scale are what matter"*。

AI policy. 本课程允许使用 AI 辅助但不允许使用 agent 直接完成 assignments，也不允许使用自动补全 (auto-complete)。在使用 AI assistant 时建议将 AI Agent Guidelines 预先输入，详细可以参考课程 AI policy guide。

Basics. 第 1-2 节课程的主要目的是能训练基础 LM。基本内容包括：tokenization, model architecture, training。

Tokenization

Overview

Tokenizer. 在原始输入(bytes) 与整数序列(tokens) 之间进行转换。

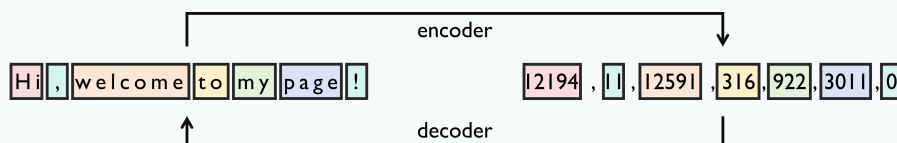


Figure 1: Tokenization

Bit, Byte and Character (Reference: UTF-8 counter, UTF-8 encoder/decoder, gpt-tokenizer playground)

计算机系统最小的信息单位为比特 (bit)，其只能取 0 或 1。字节 (byte) 是存储和处理数据的基本单位，其包含 8 个 bit，因此共有 $2^8 = 256$ 种可能状态。由于 8 位的 2 进制表示过长，因此常使用 2 位 16 进制表示 (0 ~ 9 ~ A ~ F，也是 $16^2 = 256$ 种状态，例如 $E4 = 14 \times 16^1 + 4 \times 16^0 = 228 = 11100100$)。

字符 (character) 并不能直接保存，而是需要保存为字节 (byte)。计算机会根据不同转换规则将 characters 转换为 bytes，例如在 UTF-8 规则下 你好 $\leftrightarrow$ E4 BD A0 E5 A5 BD，共 6 bytes。

Note 1. 虽然 Figure 1 中将每一个单词/符号转换成数字，但 Tokenization 处理对象并不是单独的单词而是 byte。即原始文本需要先 convert 为计算机可以读取的 byte 再使用一定的 tokenization 算法变成 token id 最终输入模型。

Note 2. Token 并不等于单词也不等于字符。一个 token 可能对应一个完整的单词如 "welcome=71326"；单词的一部分如 "warmly=[warm, ly]=[111115, 423]"；单个汉字如 "人=1995"；多个汉字的组合如 "人工智能=[人工, 智能]=[47243, 60319]"；单个汉字的一部分如 "廬=[10129, 239]"。(参考 Tiktokenizer, GPT-4o tokenization)

Note 3. 从 Efficiency 角度来看 tokenization 可以减少文本编码的长度。

Several Tokenization Methods

Compression ratio. 衡量一种 tokenization method 对于文本的压缩效率的方式是 $\text{compression ratio} = \frac{\# \text{ byte}}{\# \text{ token}}$.

Vocabulary Size. Vocabulary size 是 tokenizer 可以使用的不同 token ID 的数量.

Note 4. ① 更高的 *Compression ratio (CR)* 意味着可以使用更少的 *token (T)* 表示文本, 相同的上下文窗口也能容纳更多文字. 由于 *Transformer* 的 *self-attention* 计算量中有一部分近似为 $O(T)$, 因此高 CR 意味着更少的计算量. ② *Transformer* 中输入 *embedding* 层维度为 $V \times d$, 其中 V 为 *vocabulary size*, d 为 *hidden size*. 因此减小 *vocabulary size* 可以降低 *embedding* 层的计算量. ③ CR 与 V 实际中相互制衡, 更高的 CR 需要更大的 *vocabulary*.

Character Tokenizer. 将每个字符作为一个 token, 直接使用 Unicode 编码. (Python 中使用 `ord chr` 查看编码)

```
1 ord("a") # output will be 97
2 chr(97)  # output will be 'a'
```

Character Tokenizer 的优点是概念简单、易于理解, 但存在两个问题: ① Vocabulary size 过大, 编码所有的字符需要约 150K Unicode characters. ② 许多字符出现较少, 例如某些表情符号. ③ Unicode 编码会更新、并不稳定.

Byte Tokenizer. 在计算机中文本本身就是由 bytes 表示的, 表示方式有 UTF-8 等. Byte tokenizer 将文本先编码成 UTF-8 bytes, 再把每个 byte 作为一个 token. 因此 $\text{vocabulary size} = 256$, $\text{CR} = 1$. 虽然 Byte tokenizer 的 vocabulary size 小, 但其 compression ratio 低、表示的序列太长. (例如 你好 $\leftrightarrow$ E4 BD A0 E5 A5 BD)

Word Tokenizer. Word tokenizer 尝试将每个完整单词作为一个 token. 故相比于 character tokenizer 其 compression ratio 更高, 但存在两个重要问题: ① *Unbounded vocabulary*: 由于新的单词会不断出现, 因此词表并没有上限. ② *Out of Vocabulary (OOV)*: 类似地, 如果一个新的词并不在旧 vocabulary 中就会丢失原始信息.

Trade-off between Vocabulary Size and

当 vocabulary size 过小时, 编码文本所需要的平均 token 量就会变多, 因此在相同的上下文长度 (token 数量) 下所能处理的信息就会减少; 当 vocabulary size 过大时, 虽然编码文本所需要的平均 token 量减小但 Transformer 在预测 next token 时的 softmax 的向量维度 ($|\mathcal{V}|$) 就会变大, 会加大训练量.

GPT Tokenizer

Reference: Let's build the GPT Tokenizer from Andrej Karpathy

Byte-Pair Encoding (BPE)

Byte-Pair Encoding (BPE) [1]. 未完待续.

References

- [1] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th annual meeting of the association for computational linguistics (volume 1: long papers)*, pages 1715–1725, 2016.